S. Nithyanandam

192511007

SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA14 **COURSE NAME: Compiler Design**

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. *(BL3)*

Assessment Tool Weightage: 40%

Duration :60 Mins
On Class
Total Marks:50

QUESTIONS:5

Annexure A

DESIGN TASK

Code of Conduct:

I S. Nithyanandam / 192511007 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

S. N

1) LL(1) Predictive Parser :

1. Compute the FIRST and FOLLOW sets for all non-terminals (S, A, B, D).

2. Construct the LL(1) Predictive Parsing table using these sets

3. checks the FIRST / FIRST and FIRST / FOLLOW conflicts. If no multiple entries exists in the parsing table, the grammar is LL(1).

4. Demonstrate Predictive Parsing by showing stack, Input, and Parsing Action for a Valid input string

5. Advantages of Predictive Parser :

=) Easy to implement

=) No backtracking is required

=) Efficient syntax analysis.

=) Detects syntax errors quickly.

2) Shift - Reduce Parser :

(i) Initially the stack with $ and place the input string in the input buffer.

(ii) Perform shift operation to move symbols from the input to the stack.

(iii) When a handle is recognized, apply the appropriate reduce operation using the grammar rule.

(iv) Continue shifting and reducing until the stack contains only the start symbol

(v) If the input is completely consumed, the parser accepts the string.

(vi) shift - reduce conflicts are resolved using operator precedence and associativity.

3) Recursive Decent Parser :

* ~~Write~~ Write separate recursive procedures for S(), L(), and L'().
* Start parsing by calling S(),
* match each terminal symbol with the input.
* Continue recursively until all Productions are Processed.
* If the entire input is matched successfully the string is Accepted; Otherwise, it is Rejected.

* Include simple error handling to report invalid symbols or unexpected input.

---

4) Operator Precedence :

-> Construct an unambiguous grammar for arithmetic expressions

-> Define operator precedence :

- ✧ * has higher precedence than + and - .
- ✧ Parentheses ( ) have the highest precedence.

-> Prepare the operator precedence table using <, =, and >.

-> Use a stack - based parser to perform shift and reduce operations.

-> Parse the input id + id * id according to precedence rules.

-> When the start symbol is obtained and the input is exhausted, the string is accepted.

---

5) LALR (1) Parser :

1. Create the augmented grammar by adding S'

   -> S.

2. Compute the FIRST and FOLLOW sets.

3. Construct the canonical LR(1) item sets using Closure

and Goto operations

4. Merge states with identical cores to obtain the LALR(1) Parser.

5. Build the ACTION and GOTO Parsing tables.

6. Parse the input string by showing stack, Input, and Parsing Actions.

7. If the Parser reaches the Accept state, the input string is valid; otherwise, it is rejected